

Solution Analysis: K-th Smallest Prime Fraction

This is one of the hardest binary search problems on LeetCode because we are not binary searching on the array, we're binary searching on the answer (the fraction value). Let's build the intuition from scratch.

Source Implementation

```
class Solution {
public:
    vector<int> kthSmallestPrimeFraction(vector<int>& arr, int k) {
        int n = arr.size();
        double left = 0, right = 1, mid;
        vector<int> res;

        while (left <= right) {
            mid = left + (right - left) / 2;
            int j = 1, total = 0, num = 0, den = 0;
            double maxFrac = 0;
            for (int i = 0; i < n; ++i) {
                while (j < n && arr[i] >= arr[j] * mid) {
                    ++j;
                }

                total += n - j;

                if (j < n && maxFrac < arr[i] * 1.0 / arr[j]) {
                    maxFrac = arr[i] * 1.0 / arr[j];
                    num = i; den = j;
                }
            }

            if (total == k) {
                res = {arr[num], arr[den]};
                break;
            }

            if (total > k) {
                right = mid;
            } else {
                left = mid;
            }
        }

        return res;
    }
};
```

Problem Overview

Suppose

```
arr = [1, 2, 3, 5]
```

All possible fractions are

```
1/2 = 0.5
1/3 = 0.333
1/5 = 0.2
2/3 = 0.666
2/5 = 0.4
3/5 = 0.6
```

Sorted:

```
0.2
0.333
0.4
0.5
0.6
0.666
```

If $k = 3$, the Answer is $2/5$.

Brute Force Approach

Generate every fraction.

$$n * (n - 1) / 2 \text{ fractions}$$

Sort them. Return kth.

Complexity

```
 $O(n^2 \log n)$ 
```

Too slow.

Main Observation

Instead of generating fractions, suppose I ask you:

How many fractions are less than 0.4? Can you answer efficiently?

If yes... then we can binary search on the value. This is exactly the idea.

Binary Search on Answer Space

Every fraction lies between **0** and **1** because *numerator* < *denominator*. So the answer is somewhere in:

$$0 \text{ ----- } 1$$

We binary search this interval. Suppose *mid* = **0.5**. Now ask:

How many fractions are ≤ 0.5 ?

If *count* < *k*, it means our answer is larger. Move right.

If *count* > *k*, it means our answer is smaller. Move left.

Exactly like normal binary search.

The Counting Mechanism

But how do we count? This is the real trick.

Suppose *arr* = [**1, 2, 3, 5**] and *mid* = **0.4**. We need all fractions ≤ 0.4 .

For every numerator *arr[i]*, find all denominators *arr[j]* such that:

$$\text{arr}[i] / \text{arr}[j] \leq \text{mid}$$

Instead of dividing, multiply both sides:

$$\text{arr}[i] \leq \text{mid} * \text{arr}[j]$$

That's why the code writes `arr[i] >= arr[j] * mid` inside the while loop.

Dry Run Analysis

Given *arr* = [**1, 2, 3, 5**] and *mid* = **0.4**:

Initially *j* = **1**.

- **i = 0, numerator = 1:** Need $1 / \text{arr}[j] \leq 0.4$.
 - Check $1 / 2 = 0.5$ (Too large. Increase *j* → *j* = 2).
 - Check $1 / 3 = 0.333$ (Good). Every denominator after 3 also works ($1 / 5$ also works).
 - Count: **2 fractions**.
- **i = 1, numerator = 2:** Current *j* = 2.
 - Check $2 / 3 = 0.666$ (Too large. Move *j* → *j* = 3).
 - Check $2 / 5 = 0.4$ (Perfect).
 - Count: **1 fraction**.
- **i = 2, numerator = 3:**
 - Check $3 / 5 = 0.6$ (Too large. Move *j* → *j* = 4).
 - No denominator left.
 - Count: **0 fractions**.

Total: $2 + 1 = 3$ fractions exactly ≤ 0.4 .

Why doesn't j reset?

This is the biggest trick. Notice $1 / 3 \leq 0.4$. Now move to $2 / ?$. Numerator became larger. Fractions become larger. So denominator can only move right. Never left. Therefore, j only increases.

Total complexity is $O(n)$ instead of $O(n^2)$. This is the two-pointer optimization.

Deep Dive Code Understanding

Binary Search Mechanics

```
double left = 0; double right = 1;
```

Search interval: $0 \rightarrow 1$

```
while(left <= right)
```

Keep searching. (Practically, many implementations use an epsilon condition like `while (right - left > 1e-9)` because floating-point values rarely hit the exact answer.)

```
mid = left + (right - left) / 2;
```

Current guessed fraction value.

Variables Framework

`int j = 1;` — Current denominator pointer.

`int total = 0;` — How many fractions are $\leq mid$.

`double maxFrac = 0;` — Largest fraction $\leq mid$. This is extremely important. We'll discuss why.

Counting Logic

```
for(int i = 0; i < n; i++)
```

Treat every element as numerator.

```
while(j < n && arr[i] >= arr[j] * mid)
    j++;
```

Equivalent to $\text{arr}[i] / \text{arr}[j] > \text{mid}$. So keep moving denominator until fraction becomes $\leq \text{mid}$. Suppose j becomes 3. Then $\text{arr}[i]/\text{arr}[3]$, $\text{arr}[i]/\text{arr}[4]$, ... All are valid. Because denominator keeps increasing, fraction keeps decreasing. Hence:

```
total += n - j;
```

Why store maximum fraction?

Suppose $\text{mid} = 0.4$. Valid fractions are:

```
1/5 = 0.2  
1/3 = 0.333  
2/5 = 0.4
```

Largest valid one is $2/5$. That is the closest fraction to mid from the left. We store it.

```
if(maxFrac < arr[i] * 1.0 / arr[j])
```

Update largest fraction $\leq \text{mid}$.

```
num = i; den = j;
```

Store indices.

After Counting Step

Suppose $\text{total} = 7$ and we need $k = 5$. Meaning: Too many fractions are smaller. Answer must be smaller. So:

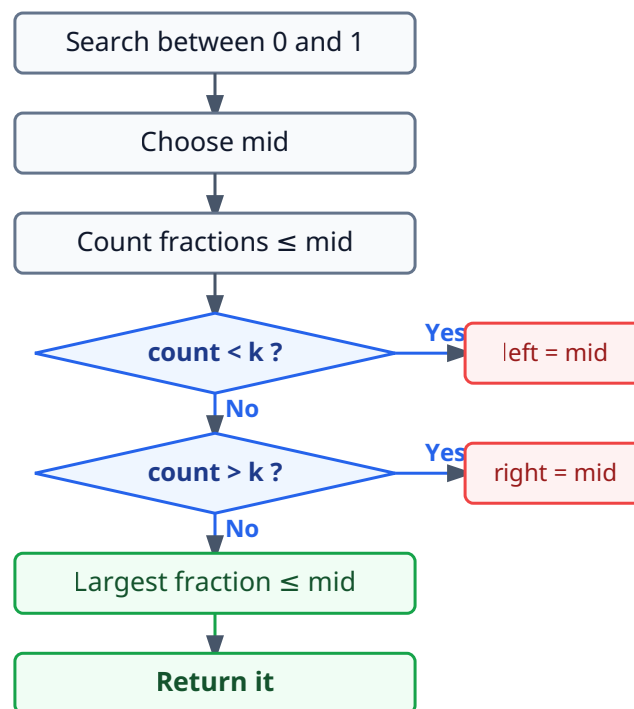
```
right = mid;
```

Suppose $\text{total} = 3$ and $k = 5$. Only three fractions are smaller. Need larger fraction. Move:

```
left = mid;
```

Why $\text{if}(\text{total} == k)$? Imagine $\text{mid} = 0.438765$. Exactly k fractions are below it. Among them, the largest one is precisely the k th smallest fraction. That is why $\text{res} = \{\text{arr}[\text{num}], \text{arr}[\text{den}]\}$; works.

Complete Execution Flow



Complexity Profile

- **Time Complexity:** Counting fractions for one mid: $O(n)$ (because j only moves forward). Binary search iterations: about 40–60 iterations to achieve sufficient floating-point precision. Total: $O(n * \log(\text{precision}))$, which is effectively $O(n)$ for practical purposes.
- **Space Complexity:** $O(1)$ (excluding the returned answer).

The heart of this algorithm is the combination of:

1. Binary search on the fraction value (answer space) instead of indices.
2. A two-pointer counting method that counts how many fractions are $\leq \text{mid}$ in linear time.
3. Tracking the largest valid fraction $\leq \text{mid}$, which becomes the answer as soon as exactly k fractions are on the left side.

Deconstructing the Core Confusion: `total += n - j;`

Query: Hence C++ `total += n - j;` can't understand this if there is j total that are valid then total will $j + 1$

This is the most common confusion in this problem. The key is understanding what j represents.

Let's focus only on this part:

```
while (j < n && arr[i] >= arr[j] * mid) {
    j++;
}
total += n - j;
```

What does j represent?

After the while loop finishes, j is the **first denominator index** that makes the fraction valid. In other words,

$$\text{arr}[i] / \text{arr}[j] \leq \text{mid}$$

is true for the first time. Everything before j is invalid. Everything from j onward is valid.

Concrete Example Walkthrough

Suppose $\text{arr} = [1, 2, 3, 5]$ and $\text{mid} = 0.4$.

Take $i = 0 \rightarrow \text{numerator} = 1$.

Initially, $j = 1$.

- Check: $j = 1 \rightarrow 1 / 2 = 0.5$. Is it ≤ 0.4 ? $\times$ **No**. So move j .
- $j = 2 \rightarrow 1 / 3 = 0.333$. Is it ≤ 0.4 ? $\checkmark$ **Yes**. Stop.

Now, $j = 2$. Which fractions are valid?

```
1/2 ×
1/3 ✓
1/5 ✓
```

Notice something: The valid denominators are:

```
Index : 2    3
Value : 3    5
```

There are $4 - 2 = 2$ valid fractions. That's why `total += n - j;` works perfectly.

Why not $j + 1$?

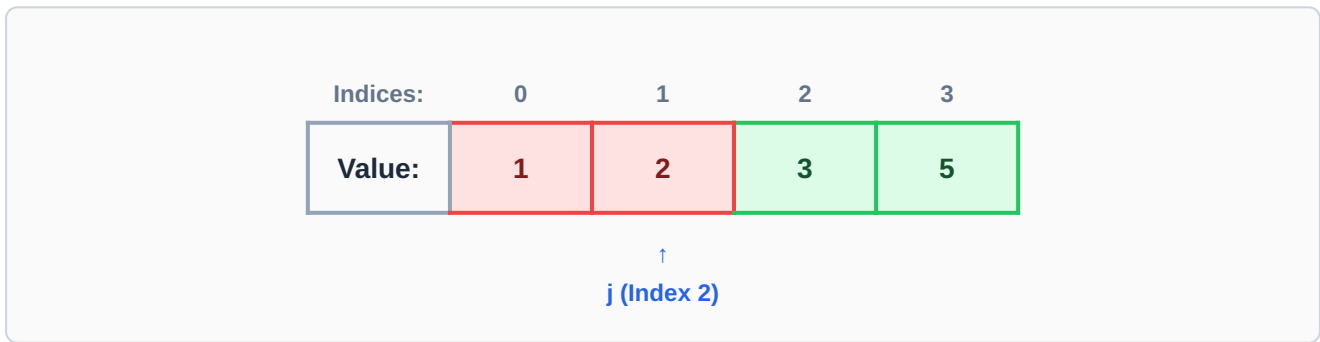
Because j is an index, not the count of valid elements. Suppose $n = 4$ and $j = 2$. The valid indices are 2 and 3. Count them: **2 elements**. Formula: $4 - 2 = 2$.

If you did $j + 1 = 3$, you'd incorrectly say there are 3 valid fractions, but there are only 2.

Another Example

Suppose $\text{arr} = [1, 2, 3, 5, 7]$ and $n = 5$. After the loop, $j = 3$. Then valid denominators are Index 3 and Index 4. Only **2** fractions. Formula: $5 - 3 = 2$. Again, not $j + 1$.

Visualizing the Window Boundary



Everything before j (Indices 0, 1) → × **Invalid fractions**

Everything from j onward (Indices 2, 3) → ✓ **Valid fractions**

So the count is simply the *number of elements from j to $n-1$* , which equals $n - j$.

A Simple Rule to Remember

After the while loop finishes:

```
0 ... j-1 → × Invalid fractions
j ... n-1 → ✓ Valid fractions
```

Therefore, `total += n - j`; is exactly the number of valid fractions for the current numerator.

If you'd like, I can also explain why the while condition is `arr[i] >= arr[j] * mid` instead of directly checking `arr[i]/arr[j] > mid`, since that's another subtle but important part of the algorithm.